



List

1. Reverse a List

- **Explanation:** Given a list, reverse the order of its elements.
- **Hint:** Use slicing (`list[::-1]`), built-in `reversed()` function, or iterate through the list in reverse order.

2. Find the Maximum and Minimum Values in a List

- **Explanation:** Determine the largest and smallest values within a given list.
- **Hint:** Use built-in functions `max()` and `min()`, or iterate through the list and compare elements.

3. Remove Duplicates from a List

- **Explanation:** Create a new list containing only unique elements from the original list.
- **Hint:** Use a `set()` to eliminate duplicates, or iterate through the list and keep track of seen elements.

4. Check if a List is Sorted

- **Explanation:** Determine if the elements in the list are in ascending order.
- **Hint:** Compare adjacent elements in the list.

5. Merge Two Sorted Lists

- **Explanation:** Given two sorted lists, create a new sorted list containing all elements from both.
- **Hint:** Use a while loop and compare elements from both lists.

6. Find the Common Elements in Two Lists

- **Explanation:** Identify the elements that appear in both lists.
- **Hint:** Use `set()` intersection (`set(list1) & set(list2)`).

7. Rotate a List by k Positions

- **Explanation:** Shift the elements of the list to the right by k positions.
- **Hint:** Use slicing to divide the list and then concatenate the parts.

8. Find the Kth Largest Element in a List

- **Explanation:** Determine the k-th largest value in the list.
- **Hint:** Sort the list and access the k-th element from the end.

9. Implement a Stack Using a List

- **Explanation:** Create a class to represent a stack data structure using a Python list.
- **Hint:** Use list methods like `append()` for `push()` and `pop()` for `pop()`.

10. Implement a Queue Using a List

- **Explanation:** Create a class to represent a queue data structure using a Python list.
- **Hint:** Use list methods like `append()` for `enqueue()` and `pop(0)` for `dequeue()`.





String

1. Reverse a String

- **Problem:** Given a string, write a function to reverse it.
- **Example:**
 - Input: "hello"
 - Output: "olleh"
- **Key Concepts:**
 - String slicing (`[::-1]`),
 - Iteration (for loop),
 - Recursion

2. Check for Palindrome

- **Problem:** Determine if a given string is a palindrome (reads the same forward and backward).
- **Example:**
 - Input: "racecar"
 - Output: True
- **Key Concepts:**
 - String slicing,
 - Iteration,
 - Two-pointer approach

Page | 2

3. Count Vowels

- **Problem:** Count the number of vowels (a, e, i, o, u) in a given string.
- **Example:**
 - Input: "hello world"
 - Output: 3
- **Key Concepts:**
 - Iteration,
 - Membership operator (`in`)

4. Remove Duplicates from String

- **Problem:** Given a string, remove all duplicate characters.
- **Example:**
 - Input: "aabbcc"
 - Output: "abc"
- **Key Concepts:**
 - Sets (for unique elements),
 - Iteration

5. Check for Anagrams

- **Problem:** Determine if two strings are anagrams of each other (contain the same





characters in a different order).

- **Example:**
 - Input: "listen", "silent"
 - Output: True
- **Key Concepts:**
 - Sorting,
 - Counters (from collections)

6. Find Longest Common Substring

- **Problem:** Find the longest common substring between two strings.
- **Example:**
 - Input: "abcdefg", "abgh"
 - Output: "ab"
- **Key Concepts:**
 - Dynamic Programming (often used for optimization)

7. Find Longest Common Prefix

- **Problem:** Find the longest common prefix among a list of strings.
- **Example:**
 - Input: ["flower", "flow", "flight"]
 - Output: "fl"
- **Key Concepts:**
 - Iteration,
 - String comparison

8. String Permutations

- **Problem:** Generate all possible permutations of a given string.
- **Example:**
 - Input: "abc"
 - Output: ["abc", "acb", "bac", "bca", "cab", "cba"]
- **Key Concepts:**
 - Recursion,
 - Backtracking

9. Check for Substring

- **Problem:** Determine if a given string is a substring of another string.
- **Example:**
 - Input: "hello", "llo"
 - Output: True
- **Key Concepts:**
 - String methods like `find()` or `index()`

10. String Formatting

- **Problem:** Format a string according to given specifications (e.g., using f-strings or the





`format()` method).

- **Example:**
 - Input: name = "Alice", age = 30
 - Output: "Hello, my name is Alice and I am 30 years old."
- **Key Concepts:**
 - String formatting techniques

Dict

1. Given a list of words, find the frequency of each word using a dictionary.

- **Example:**
Python

```
words = ["apple", "banana", "apple", "cherry", "banana", "apple"]
```
- **Expected Output:**
Python

```
{'apple': 3, 'banana': 2, 'cherry': 1}
```

2. Given two dictionaries, merge them into a new dictionary.

- **Example:**
Python

```
dict1 = {'a': 1, 'b': 2}  
dict2 = {'c': 3, 'd': 4}
```
- **Expected Output:**
Python

```
{'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

3. Given a dictionary, find the key with the maximum value.

- **Example:**
Python

```
my_dict = {'x': 5, 'y': 2, 'z': 9}
```
- **Expected Output:**
Python

```
'z'
```

4. Given a dictionary, find the sum of all values.

- **Example:**
Python

```
my_dict = {'a': 10, 'b': 20, 'c': 30}
```
- **Expected Output:**
Python





60

5. Given a dictionary, check if a given key exists in the dictionary.

- **Example:**

Python

```
my_dict = {'name': 'Alice', 'age': 30}
key_to_check = 'age'
```

- **Expected Output:**

Python

True

6. Given a dictionary, remove a specific key-value pair.

- **Example:**

Python

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
key_to_remove = 'b'
```

- **Expected Output:**

Python

```
{'a': 1, 'c': 3}
```

Page | 5

7. Given a dictionary, create a new dictionary with only the keys.

- **Example:**

Python

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
```

- **Expected Output:**

Python

```
['a', 'b', 'c']
```

8. Given a dictionary, create a new dictionary with only the values.

- **Example:**

Python

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
```

- **Expected Output:**

Python

```
[1, 2, 3]
```

9. Given a dictionary, sort it by keys.

- **Example:**





Python

```
my_dict = {'c': 3, 'a': 1, 'b': 2}
```

- **Expected Output:**

Python

```
{'a': 1, 'b': 2, 'c': 3}
```

10. Given a dictionary, sort it by values.

- **Example:**

Python

```
my_dict = {'c': 1, 'a': 3, 'b': 2}
```

- **Expected Output:**

Python

```
{'c': 1, 'b': 2, 'a': 3}
```



Problem Definition:**Student Management System****Objective:**

To create a basic student management system using Python, utilizing lists and dictionaries to store and manage student data. The system should provide a user-friendly menu with the following options:

1. **Add Student:**
 - Allow users to add new students with their details (e.g., name, roll number, age, contact).
 - Store student data in a list of dictionaries.
2. **View Students:**
 - Display all students with their details.
3. **Search Student:**
 - Allow users to search for a specific student by roll number or name.
4. **Update Student:**
 - Enable users to update the details of an existing student.
5. **Delete Student:**
 - Allow users to delete a student from the system.
6. **Exit:**
 - Terminate the program.

Page | 7

Data Structures:

- Use a list to store all student records.
- Each student record will be represented as a dictionary containing the following keys:
 - roll_no
 - name
 - age
 - contact

Instructions:

1. **Create a list named students to store student data.**
2. **Implement a function for each menu option:**
 - **add_student():**
 - Prompt the user to enter student details.
 - Create a dictionary for the new student and append it to the students list.
 - **view_students():**
 - Iterate through the students list and display the details of each student.
 - **search_student():**
 - Allow the user to search by roll number or name.
 - Iterate through the students list to find the matching student(s) and display their details.
 - **update_student():**
 - Allow the user to search for a student by roll number.
 - If found, prompt the user for updated details and modify the existing dictionary.





- **delete_student():**
 - Allow the user to search for a student by roll number.
 - If found, remove the corresponding dictionary from the students list.
- 3. **Implement a main function:**
 - Display the main menu with the available options.
 - Continuously prompt the user for their choice until they select "Exit."
 - Call the appropriate function based on the user's choice.

Sample Input/Output:

Input:

```
1
Enter Roll No: 101
Enter Name: Alice
Enter Age: 20
Enter Contact: 1234567890
1
Enter Roll No: 102
Enter Name: Bob
Enter Age: 21
Enter Contact: 9876543210
2
3
Enter Roll No: 101
2
Enter Roll No: 103
4
Enter Roll No: 101
Enter Name: Alice
Enter Age: 20
Enter Contact: 9876543210
5
Enter Roll No: 102
2
6
```

Output:

```
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
```





Enter your choice: 1
Enter Roll No: 101
Enter Name: Alice
Enter Age: 20
Enter Contact: 1234567890

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Enter your choice: 1
Enter Roll No: 102
Enter Name: Bob
Enter Age: 21
Enter Contact: 9876543210

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Enter your choice: 2
Roll No: 101, Name: Alice, Age: 20, Contact: 1234567890
Roll No: 102, Name: Bob, Age: 21, Contact: 9876543210

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Enter your choice: 3
Enter Roll No: 101
Roll No: 101, Name: Alice, Age: 20, Contact: 1234567890

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Enter your choice: 3
Enter Roll No: 103





No student found.

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Enter your choice: 4
Enter Roll No: 101
Enter Name: Alice
Enter Age: 20
Enter Contact: 9876543210

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Enter your choice: 5
Enter Roll No: 102

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Enter your choice: 2
Roll No: 101, Name: Alice, Age: 20, Contact: 9876543210

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Enter your choice: 6

